Auction System — Capstone Project Report

CS2043 · TamTamCatWorks

Team TamTamCatWorks

2026-06-04

Table of contents

1 The Problem & Scope of Implementation

An online **bidding system** is a platform that allows multiple users to compete for products or services within a determined time window. Unlike traditional fixed-price commerce, a seller lists their product on the system and the final price is decided through a competitive bidding process among registered bidders.

Our capstone project implements a full-stack, real-time auction platform with:

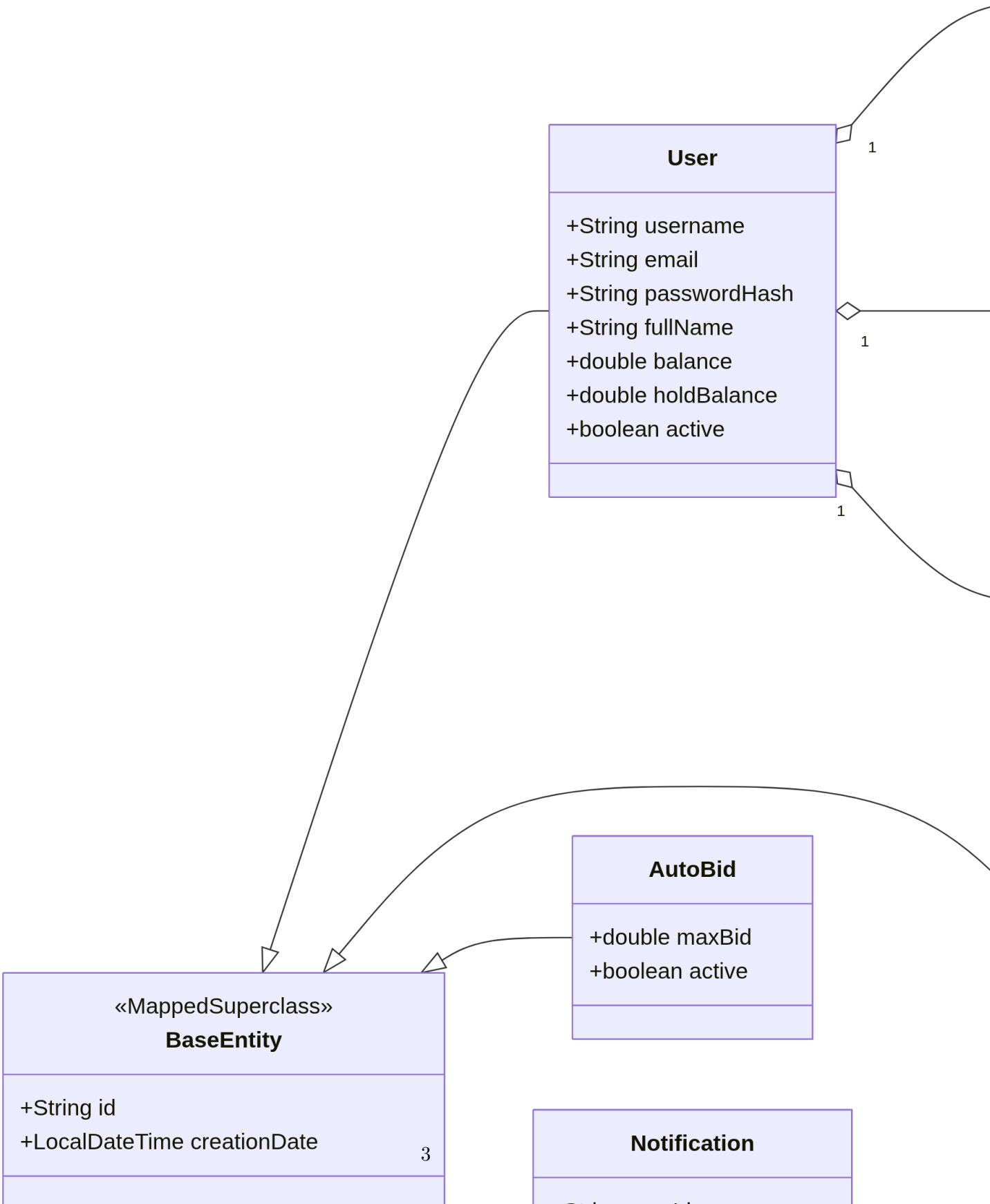
- **User management** with role-based access (Buyer, Seller, Admin)
- **Auction lifecycle management** (creation → activation → bidding → closure/cancellation)
- **Real-time updates** via WebSocket (STOMP over SockJS)
- **Concurrency-safe bidding** with optimistic locking and fund reservation
- **Advanced features**: anti-sniping, auto-bidding, search & pagination, object storage with MinIO
- A rich **JavaFX desktop client** styled with AtlantaFX

The system is built as a **multi-module Maven project** with a clear separation of concerns across six modules: `shared`, `model`, `persist`, `service`, `api`, and `client`.

2 System Design

2.1 Main Classes

The domain model is anchored by an abstract `BaseEntity` superclass that every persistent entity extends. The class diagram below shows the core relationships:


User
+String username
+String email
+String passwordHash
+String fullName
+double balance
+double holdBalance
+boolean active
1
1
1
AutoBid
+double maxBid
+boolean active
«MappedSuperclass»
BaseEntity
+String id
+LocalDateTime creationDate
3
Notification

Key design decisions:

- `BaseEntity` is annotated with `@MappedSuperclass` and uses `@GeneratedValue(strategy = GenerationType.UUID)` for ID generation, ensuring globally unique identifiers without database coordination.
- `@PrePersist` automatically stamps `creationDate` on initial save (`BaseEntity.java:42-45`).
- `Auction` owns a `@Version` field (`Long version`) for JPA optimistic locking — this is the primary mechanism for concurrency-safe bidding.

2.2 OOP Principles

2.2.1 Inheritance

The project uses a deep, well-structured class hierarchy:

```
BaseEntity (abstract, @MappedSuperclass)
   User
   Auction
   BidTransaction
   AutoBid
   Notification
   Item (abstract, @Entity with JOINED inheritance)
       Art
       Electronics
       Vehicle
       Other
```

`Item` uses JPA's **JOINED inheritance strategy** (`@Inheritance(strategy = InheritanceType.JOINED)`) with a discriminator column (`@DiscriminatorColumn(name = "item_type")`), meaning each subclass maps to its own database table that joins back to the parent `items` table. This was chosen because each item category has **different domain-specific columns** (e.g., `Art` has `artist`, `yearCreated`, `medium`; `Vehicle` has its own fields) which would be wasteful as nullable columns in a single table.

```java
// Item.java - Abstract base with JOINED inheritance
@Entity
@Table(name = "items")
@Inheritance(strategy = InheritanceType.JOINED)
@DiscriminatorColumn(name = "item_type")
public abstract class Item extends BaseEntity { ... }
```

```java
// Art.java - Concrete subclass with its own table
@Entity
@Table(name = "art_items")
@DiscriminatorValue("ART")
@PrimaryKeyJoinColumn(name = "item_id")
```

```
public class Art extends Item {
    private String artist;
    private int yearCreated;
    private String medium;
    private String dimensions;
    private boolean hasCertificate;
    ...
}
```

2.2.2 Abstraction

`BaseEntity` is an `abstract class` — it cannot be instantiated directly. It defines shared identity and lifecycle fields (`id`, `creationDate`) and a `protected` constructor, forcing all subclasses to go through their own constructors. The `Item` class further declares:

```
public abstract String getSpecificInfo();
```

Each concrete item must override this method to provide category-specific display text. This is a clean application of the **Template Method** pattern — the superclass defines what to do, subclasses define how.

2.2.3 Encapsulation

Domain objects enforce **business invariants** through their setters. For example, `Item` validates that names are non-blank and starting prices are positive:

```
// Item.java - Setter with validation
public void setName(String name) {
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Item name must not be empty.");
    }
    this.name = name.trim();
}
```

Similarly, `User` protects balance integrity through a suite of fund management methods (`holdFunds`, `releaseHeldFunds`, `consumeHeldFunds`) that validate positive amounts and check for sufficient balances before mutating state.

`Auction.getBidHistory()` returns `Collections.unmodifiableList(bidHistory)` — exposing the list as read-only prevents external code from bypassing the `recordBid()` method, which is the only sanctioned mutation path.

2.2.4 Polymorphism

Polymorphism appears in several forms:

1. **Method overriding**: every entity overrides `toString()` from `BaseEntity/Object` with a display format specific to its type.
2. **Abstract method dispatch**: `Item.getSpecificInfo()` resolves at runtime to the concrete subclass's implementation (e.g., `Art.getSpecificInfo()` returns artist and medium info).
3. **Interface-based dispatch**: `ItemCreator` subclasses override `buildItem()` and `supports()`, allowing the `ItemService` to dynamically route creation to the correct factory.

2.3 Design Patterns

2.3.1 Factory Pattern — `ItemCreator` Hierarchy

Item creation uses a **Factory Method** pattern combined with a **Registry**. The abstract `ItemCreator` class defines a template:

```
// ItemCreator.java - Template + Factory Method
public abstract class ItemCreator {
    public Item create(..., Map<String, Object> details) {
        validate(...);                    // common validation
        return Objects.requireNonNull(
            buildItem(..., details),      // subclass factory method
            "buildItem() must not return null"
        );
    }
    protected abstract Item buildItem(...);
    public abstract ItemType supports();  // identifies which type this factory handles
}
```

Concrete creators (`ArtCreator`, `ElectronicsCreator`, `VehicleCreator`, `OtherCreator`) are Spring `@Component` beans. They are auto-discovered and registered in `ItemService` at startup:

```
// ItemService.java - Registry pattern via @PostConstruct
@PostConstruct
void init() {
    registry = new HashMap<>();
    for (ItemCreator creator : creators) {   // Spring injects all ItemCreator beans
        registry.put(creator.supports(), creator);
    }
}
```

This design is **open for extension**: adding a new item category requires only a new `Item` subclass and a corresponding `ItemCreator` — no modification to `ItemService` or `ItemCreator`.

2.3.2 Observer Pattern — Spring Application Events

The system implements the Observer pattern using **Spring's `ApplicationEventPublisher`**. Domain events are modeled as immutable Java records:

```
// BidEvent.java - Published when a bid is accepted
public record BidEvent(
    String auctionId, String auctionTitle,
    String sellerId, String bidderId,
    String previousLeaderId, double amount
) {}
```

```
// AuctionEvent.java - Published on status changes
public record AuctionEvent(
    String auctionId, String auctionTitle,
    String sellerId, String leadingBidderId,
    double currentPrice, AuctionStatus newStatus,
    String reason
) {}
```

The **publisher** is `BidService` and `AuctionService`. Multiple **listeners** react independently:

Listener	Annotation	Responsibility
`NotificationService`	`@EventListener`	Persists notification records to DB
`NotificationBroadcastListener`	`@EventListener`	Pushes real-time WebSocket messages
`AutoBidService`	`@TransactionalEventListener(AFTER_COMMIT)`	Triggers counter-bidding auto-bids
`UserStateBroadcastListener`	`@EventListener`	Pushes updated user balance/state

This architecture fully decouples the bidding core from notification, broadcast, and auto-bid logic.

2.3.3 Singleton Pattern (via Spring IoC)

All `@Service`, `@Component`, and `@Configuration` beans are Spring-managed **singletons** by default. This means `BidService`, `AuctionService`, `NotificationService`, etc. each have exactly one instance per application context, shared across all request-handling threads. Spring's IoC container manages lifecycle, dependency injection, and thread-safety guarantees through its proxying mechanism.

2.3.4 Builder Pattern — MapStruct Mappers

DTO-to-entity and entity-to-DTO conversions use **MapStruct** (compile-time code generation). The `AuctionMapper` interface declares mappings declaratively:

```
@Mapper(componentModel = "spring", uses = ItemMapper.class)
public interface AuctionMapper {
```

```
    @Mapping(target = "sellerId", source = "seller.id")
    @Mapping(target = "sellerName", source = "seller.fullName")
    @Mapping(target = "itemId", source = "item.id")
    // ... additional mappings
    AuctionResponse toResponse(Auction auction);
}
```

MapStruct generates builder-style mapping code at compile time, avoiding reflection-based mapping overhead at runtime.

3 The Architecture — Modules, Layers, Tech Stack

3.1 Client-Server Architecture & Tech Stack

The application follows a **client-server architecture**:

Component	Technology	Role
Server (API)	Spring Boot 3.2.5, Java 21	REST API + WebSocket broker
Client	JavaFX 21 + AtlantaFX	Desktop GUI
Database	PostgreSQL 16	Persistent storage
Object Storage	MinIO (S3-compatible)	Auction item images
Containerization	Docker Compose	Dev environment orchestration

The server exposes RESTful endpoints under `/auctions`, `/users`, `/items`, `/notifications`, and `/admin`. The client communicates via HTTP (using Java's `HttpClient`) and STOMP over SockJS for real-time updates.

3.2 MVC Application

3.2.1 Server MVC

The server follows a clean **layered MVC architecture**:

```
Shared Module (request/response DTOs - Java records)

API Module   Service Module  Persist Module  Model Module
Controllers  Business Logic  Repositories    JPA Entities
WebSocket    Mappers         (Spring Data)
Exception    Events
Security     Image Storage
```

Request flow:

1. Client sends HTTP request → `@RestController` (e.g., `AuctionController`)
2. Controller extracts session user, delegates to `@Service` (e.g., `AuctionService`)
3. Service applies business logic, uses `@Repository` (e.g., `AuctionRepository`) for persistence
4. Response mapped back via MapStruct → returned as JSON

```
// AuctionController.java - REST endpoint example
@RestController
@RequestMapping("/auctions")
public class AuctionController {
    @PostMapping
    public ResponseEntity<AuctionResponse> create(
            @RequestBody CreateAuctionRequest req, HttpSession session) {
        String sellerId = requireUserId(session);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(auctionService.createByRequest(sellerId, req));
    }
}
```

3.2.2 Client MVC

The JavaFX client uses **FXML for views**, **controller classes**, and the **shared module** as the model layer:

- **View (FXML)**: declarative XML layouts under `client/src/main/resources/fxml/`
- **Controller**: annotated with `@Route` (custom annotation for navigation routing) and `@FXML`-injected UI components
- **Model**: `shared` module records (`AuctionResponse`, `BidResponse`, `UserResponse`, etc.)

The `BaseController` class provides common utility (error display, API client access), and the `Navigation` class handles FXML-based routing with context data passing.

3.3 Integration with Maven

The project uses a **Maven multi-module POM** structure where the root `pom.xml` defines six child modules:

```
<modules>
    <module>shared</module>   <!-- Request/Response DTOs -->
    <module>model</module>    <!-- JPA entities -->
    <module>persist</module>  <!-- Repositories -->
    <module>service</module>  <!-- Business logic -->
    <module>api</module>      <!-- REST + WebSocket -->
    <module>client</module>   <!-- JavaFX desktop app -->
</modules>
```

Key Maven integrations:

Plugin / Tool	Purpose
`spring-boot-starter-parent`	Dependency & plugin version management
`maven-compiler-plugin`	Java 21 compilation with MapStruct annotation processing
`jacoco-maven-plugin`	Code coverage reporting
`maven-checkstyle-plugin`	Google Java Style enforcement
`maven-release-plugin`	Semantic versioning & tag-based releases
`maven-surefire-plugin`	Unit test execution

The `dependencyManagement` section ensures all modules use consistent versions via `${project.version}`, and the MapStruct processor is configured as an annotation processor path in the compiler plugin.

3.4 Unit Testing with JUnit

The project has comprehensive test coverage across the `service`, `api`, and `persist` modules using **JUnit 5** and **Mockito**:

Module	Test Files	Focus Area
`service`	`BidServiceTest`, `AuctionServiceTest`, `AuctionSchedulerTest`, `UserServiceTest`, `ItemServiceTest`, `NotificationServiceTest`, `ImageStorageServiceTest`, `AuthUserDetailsServiceTest`, `BidMapperTest`, `AuctionMapperTest`	Business logic, mappers
`api`	`AuctionControllerTest`, `UserControllerTest`, `ItemControllerTest`, `NotificationControllerTest`, `JpaConnectionTest`	HTTP endpoints, integration
`persist`	`AuctionRepositoryTest`, `BidTransactionRepositoryTest`, `UserRepositoryTest`, `NotificationRepositoryTest`	Repository queries

Tests use `@ExtendWith(MockitoExtension.class)` with `@Mock` dependencies and `ReflectionTestUtils` for setting internal state. Example from `BidServiceTest`:

```java
@ExtendWith(MockitoExtension.class)
class BidServiceTest {
    @Mock private AuctionRepository auctionRepository;
    @Mock private BidTransactionRepository bidRepository;
    @Mock private UserRepository userRepository;
    @Mock private ApplicationEventPublisher eventPublisher;

    @Test
    void testPlaceBidSuccess() {
        auction.open();
        BidRequest request = new BidRequest(120.0, "MANUAL");
        when(auctionRepository.findById("auc123"))
            .thenReturn(Optional.of(auction));
        when(userRepository.findById("bidder123"))
            .thenReturn(Optional.of(bidder));
        // ...
        BidResponse response = bidService.placeBid("auc123", "bidder123", request);
        assertNotNull(response);
        assertEquals(120.0, response.amount());
        assertEquals(120.0, bidder.getHoldBalance());
        verify(auctionRepository, times(1)).save(auction);
        verify(eventPublisher, times(1)).publishEvent(any(BidEvent.class));
    }
}
```

Anti-sniping behavior is also tested — verifying that a bid placed within the snipe window extends the auction end time.

3.5 CI/CD with GitHub Actions

Two GitHub Actions workflows automate the build-test-release pipeline:

3.5.1 CI Workflow (`ci-server.yaml`)

Triggered on pushes to `main`/`api` branches and all pull requests:

```yaml
name: CI
on:
  push:
    branches: [main, api]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - name: Build, test, and check style
        working-directory: auction-system
        run: mvn -B -V clean verify
      - name: Verify API fat jar
        run: |
          JAR=$(find api/target -name "api-*.jar" | head -1)
          jar tf "$JAR" | grep -q 'BOOT-INF/classes/'
```

The `mvn clean verify` phase executes: **compilation → checkstyle → unit tests → JaCoCo coverage → fat JAR packaging**. The final step validates the Spring Boot fat JAR was correctly assembled.

3.5.2 Release Workflow (`release.yaml`)

Triggered by version tags (`v*`) and manual dispatch:

```
name: Release
on:
  push:
    tags: ['v*']
  workflow_dispatch:

jobs:
  release:
    steps:
      - run: mvn clean package -DskipTests
      - uses: softprops/action-gh-release@v2
        with:
          files: auction-system/api/target/api-*.jar
```

This builds the release JAR (skipping tests — already validated by CI) and publishes it as a GitHub Release artifact.

4 Specific Functionalities

4.1 User / Item Management

4.1.1 User Management

Users are managed through `UserService` with the following capabilities:

- **Registration**: validates unique username and email, hashes password with `BCryptPasswordEncoder`, auto-creates `BuyerProfile` and `SellerProfile`.
- **Login**: verifies credentials against the BCrypt hash. Sessions are managed via Spring Security's `HttpSession`.
- **Balance Management**: `holdFunds()` / `releaseHeldFunds()` / `consumeHeldFunds()` implement a **two-phase balance reservation** model for safe concurrent bidding.
- **Admin Promotion**: users can be promoted to admin with specific permissions, with all actions logged to `AdminProfile.actionLog`.
- **Account Suspension**: admins can suspend/activate user accounts.

```
// UserService.java - Registration
@Transactional
public User register(String username, String email, String password, String fullName) {
    validateRegistrationInputs(username, email);  // uniqueness checks
    User user = new User(username, email,
        passwordEncoder.encode(password), fullName, 0.0);
    user.setBuyerProfile(new BuyerProfile());
    user.setSellerProfile(new SellerProfile());
    return userRepository.save(user);
}
```

Security configuration (`WebSecurityConfig.java`) uses Spring Security with:

- `BCryptPasswordEncoder` for password hashing
- `DaoAuthenticationProvider` with custom `UserDetailsService`
- Session-based authentication (`SessionCreationPolicy.IF_REQUIRED`)
- Role-based access: `/admin/**` requires `ROLE_ADMIN`

4.1.2 Auction Item Management

Items are created through the **Factory pattern** described earlier. The `ItemService` serves as the entry point:

1. Client sends `ItemRequest` with `itemType` (e.g., `"ART"`, `"ELECTRONICS"`)
2. `ItemService` looks up the correct `ItemCreator` from its registry
3. The creator validates category-specific fields and constructs the entity
4. The entity is persisted via `ItemRepository`

The `CreateAuctionRequest` endpoint combines item creation and auction creation in a single transaction via `AuctionService.createWithItem()`.

4.2 Auction Functionalities

4.2.1 Bidding

The `BidService.placeBid()` method is the core of the bidding engine. It operates within a `@Transactional` boundary and implements the following logic:

```
1. Fetch auction and bidder from DB
2. Validate: auction is ACTIVE, bid amount exceeds (currentPrice + minimumIncrement)
3. Calculate required balance (handle re-bidding against self)
4. Hold bidder's funds (move from balance → holdBalance)
5. Release previous leader's held funds (if outbid)
6. Record the bid on the auction entity
7. Check anti-snipe window → extend end time if needed
8. Persist all changes
9. Publish BidEvent (triggers notifications, WebSocket broadcast, auto-bid cascade)
10. Publish UserStateEvent (triggers balance update push to client)
```

```java
// BidService.java - Fund reservation logic
if (rebiddingAgainstSelf) {
    bidder.holdFunds(requiredAvailableBalance);  // only hold the delta
} else {
    bidder.holdFunds(request.amount());
    if (previousLeader != null) {
        previousLeader.releaseHeldFunds(currentPrice);  // refund outbid user
        userRepository.save(previousLeader);
    }
}
```

4.2.2 Auction Closure & Management with Spring Scheduler

The `AuctionScheduler` runs as a Spring `@Scheduled` component that polls every 30 seconds for expired auctions:

```java
@Scheduled(fixedRate = 30_000)
@Transactional
public void closeExpiredAuctions() {
    List<Auction> expired = auctionService.findByStatus(AuctionStatus.ACTIVE)
        .stream()
        .filter(a -> a.getEndTime().isBefore(LocalDateTime.now()))
        .toList();

    for (Auction auction : expired) {
        auctionService.close(auction.getId());
    }
}
```

On closure, `AuctionService.close()`:

1. Calls `auction.close()` (state transition `ACTIVE → CLOSED`)
2. Consumes the winner's held funds (`leadingBidder.consumeHeldFunds(currentPrice)`)
3. Deposits the sale amount into the seller's account (`seller.addBalance(currentPrice)`)
4. Publishes `AuctionEvent` and `UserStateEvent` for real-time notification

On cancellation, held funds are released back to the leading bidder instead.

4.2.3 Global Exception Handling with `@RestControllerAdvice`

The `GlobalExceptionHandler` provides a centralized, consistent error response format across all REST endpoints:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(NoSuchElementException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(errorBody(ex.getMessage(), HttpStatus.NOT_FOUND));
    }

    @ExceptionHandler(IllegalArgumentException.class)    // → 400
    @ExceptionHandler(AuthenticationException.class)      // → 401
    @ExceptionHandler(IllegalStateException.class)        // → 409
    // Each maps to an appropriate HTTP status code

    private Map<String, Object> errorBody(String message, HttpStatus status) {
        return Map.of(
            "timestamp", LocalDateTime.now(),
            "status", status.value(),
            "error", status.getReasonPhrase(),
            "message", message
        );
    }
}
```

This design means service-layer code can simply throw standard Java exceptions (`IllegalArgumentException`, `NoSuchElementException`, `IllegalStateException`) and the framework automatically translates them to the correct HTTP status codes and a uniform JSON error body.

4.2.4 GUI with JavaFX & AtlantaFX

The desktop client is a rich JavaFX application using **AtlantaFX** for modern, Material-Design-inspired styling. Key architectural decisions:

- **FXML-based views**: 20+ FXML files for layouts (login, register, dashboard, auction list, auction detail, admin panels, etc.)
- **Custom `@Route` annotation**: controllers declare their FXML path and layout, and the `Navigation` class handles routing:

```
@Route(fxml = "/fxml/auction-detail.fxml", layout = Route.DASHBOARD_LAYOUT)
public class AuctionDetailController extends BaseController { ... }
```

- **`BaseController`**: provides shared API client access, error display, and loading state management
- **`AsyncTask`**: utility for running background work off the JavaFX thread, then marshalling results back to the UI thread via `Platform.runLater()`
- **`SessionManager`**: manages authentication state and WebSocket subscriptions globally

4.3 Concurrency & Real-time Updates

4.3.1 Concurrent Bidding Handling

Concurrent bidding is handled through a **multi-layer concurrency strategy**:

Layer 1 — JPA Optimistic Locking (`@Version`):

```
// Auction.java
@Version
private Long version;
```

When two transactions try to update the same auction simultaneously, JPA detects the version mismatch on the second commit and throws `OptimisticLockException`. Spring's `@Transactional` rolls back the loser's transaction, ensuring only one bid succeeds at a time.

Layer 2 — `synchronized` methods on Auction:

The domain model adds `synchronized` on all state-mutating methods (`open()`, `close()`, `cancel()`, `recordBid()`, `extendEndTime()`) as a defense-in-depth measure:

```
public synchronized void recordBid(BidTransaction tx) {
    bidHistory.add(tx);
    currentPrice = tx.getAmount();
    leadingBidder = tx.getBidder();
}
```

Layer 3 — `@Transactional` boundaries:

All service methods that modify state run within Spring-managed transactions. The `BidService.placeBid()` method wraps the entire bid flow — validation, fund reservation, bid recording, and event publishing — in a single atomic transaction.

Layer 4 — Two-phase balance reservation:

Rather than directly deducting balance, the system **holds funds** when a bid is placed and only **consumes** them on auction closure. If the user is outbid, held funds are **released** back to their available balance. This prevents double-spending even under concurrent bid attempts.

4.3.2 Real-time Updates (Observer / WebSocket)

The real-time notification pipeline uses **Spring Application Events** as the internal pub-sub mechanism and **STOMP over SockJS** as the transport to clients.

Server-side architecture:

```
BidService.placeBid()

      publishes BidEvent
         → NotificationService        (persists to DB)
         → NotificationBroadcastListener (pushes via WebSocket)
         → AutoBidService             (triggers competing auto-bids)

      publishes UserStateEvent
         → UserStateBroadcastListener  (pushes balance update via WebSocket)
```

WebSocket configuration:

```java
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic", "/queue");  // broadcast + user-private
        config.setApplicationDestinationPrefixes("/app");
        config.setUserDestinationPrefix("/user");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }
}
```

Topic structure:

Topic	Payload	Scope
`/topic/auctions/{id}`	`AuctionPriceUpdate`	Broadcast to all viewers
`/topic/auctions/{id}/status`	`AuctionStatusUpdate`	Broadcast on status change
`/user/queue/notifications`	`NotificationResponse`	Private to specific user

Topic	Payload	Scope
`/topic/user-state/{userId}`	`UserResponse`	User balance refresh

Client-side subscription (`AuctionWebSocketClient.java`):

```
public StompSession.Subscription subscribeToPrice(
        String auctionId, Consumer<AuctionPriceUpdate> onPriceUpdate) {
    return session.subscribe("/topic/auctions/" + auctionId,
        new StompFrameHandler() {
            @Override
            public Type getPayloadType(StompHeaders headers) {
                return AuctionPriceUpdate.class;
            }
            @Override
            public void handleFrame(StompHeaders headers, Object payload) {
                if (payload instanceof AuctionPriceUpdate update) {
                    onPriceUpdate.accept(update);
                }
            }
        });
}
```

The `AuctionDetailController` subscribes to both price and status topics on view load, and updates the UI in real-time via `Platform.runLater()`. Subscriptions are automatically cleaned up when the scene is detached.

4.4 Further Functionalities

4.4.1 Anti-Sniping

Sniping — placing a bid in the last seconds of an auction to prevent counter-bids — is countered by an **automatic time extension** mechanism.

The `AntiSnipeProperties` record (loaded from `application.yaml`) defines:

- `windowSeconds` — how close to the end time a bid must be to trigger extension (default: 60s)
- `extensionSeconds` — how many seconds to extend the auction by (default: 120s)

```
@ConfigurationProperties(prefix = "auction.anti-snipe")
public record AntiSnipeProperties(int windowSeconds, int extensionSeconds) {}
```

The check is embedded in `BidService.placeBid()`:

```
LocalDateTime snipeWindowStart = auction.getEndTime()
    .minusSeconds(antiSnipe.windowSeconds());
```

```
if (!LocalDateTime.now().isBefore(snipeWindowStart)) {
    auction.extendEndTime(antiSnipe.extensionSeconds());
}
```

This ensures that any bid placed within the anti-snipe window automatically extends the auction, giving other participants a fair chance to respond.

4.4.2 Auto-Bidding

The `AutoBidService` implements an **automated proxy bidding** system. When a user registers an auto-bid with a maximum price, the system bids on their behalf whenever they are outbid.

Algorithm:

1. A manual bid triggers a `BidEvent`
2. `AutoBidService.onBidPlaced()` listens with `@TransactionalEventListener(phase = AFTER_COMMIT)` — ensuring the previous bid's transaction is fully committed
3. Active auto-bids for the auction are loaded (excluding the current leader)
4. Candidates are ranked in a `PriorityQueue` sorted by `maxBid` descending
5. The top candidate places an automatic bid at `currentPrice + minimumIncrement`
6. If the bid exceeds their `maxBid`, their auto-bid is deactivated
7. The new bid triggers another `BidEvent`, cascading the process

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void onBidPlaced(BidEvent event) {
    // ... load candidates, build PriorityQueue
    AutoBid best = queue.poll();
    double nextBid = auction.getCurrentPrice() + auction.getMinimumIncrement();

    if (nextBid > best.getMaxBid()) {
        best.deactivate();  // can't afford to bid
        return;
    }

    bidService.placeBid(event.auctionId(), best.getBidder().getId(),
        new BidRequest(nextBid, "AUTO"));  // triggers cascade
}
```

The use of `REQUIRES_NEW` propagation ensures each auto-bid runs in its own transaction, preventing cascading rollbacks and making the concurrency behavior predictable.

4.4.3 Bid History Visualization with Line Chart

The `AuctionDetailController` renders bid history as a **JavaFX `LineChart`** showing bid amounts over bid sequence:

```
private void populateBidChart(List<BidResponse> bids) {
    XYChart.Series<Number, Number> series = new XYChart.Series<>();
    series.setName("Bid amount");

    int bidIndex = 1;
    for (BidResponse bid : bids) {
        series.getData().add(new XYChart.Data<>(bidIndex++, bid.amount()));
    }

    bidHistoryChart.getData().clear();
    bidHistoryChart.getData().add(series);
    // Configure axes with appropriate bounds
}
```

The chart auto-scales its axes, uses bid sequence number on the X-axis and dollar amount on the Y-axis, giving users an immediate visual sense of bidding momentum.

4.4.4 Search & Pagination

The `AuctionRepository` implements search with **JPQL joins** and Spring Data's `Pageable`:

```
@Query("""
    SELECT DISTINCT a FROM Auction a
    JOIN a.item i JOIN a.seller s
    WHERE (:status IS NULL OR a.status = :status)
      AND (:keyword IS NULL OR :keyword = ''
        OR LOWER(a.title) LIKE LOWER(CONCAT('%', :keyword, '%'))
        OR LOWER(i.name)  LIKE LOWER(CONCAT('%', :keyword, '%'))
        OR LOWER(i.description) LIKE LOWER(CONCAT('%', :keyword, '%'))
        OR LOWER(s.fullName) LIKE LOWER(CONCAT('%', :keyword, '%'))
      )
      AND (:itemType IS NULL OR TYPE(i) = :itemType)
    ORDER BY a.creationDate DESC
    """)
Page<Auction> searchPaged(@Param("keyword") String keyword,
                          @Param("status") AuctionStatus status,
                          @Param("itemType") Class<? extends Item> itemType,
                          Pageable pageable);
```

Key features:

- **Full-text-like search** across auction title, item name, item description, and seller name
- **Category filtering** using JPA's `TYPE()` operator for polymorphic item type discrimination
- **Server-side pagination** via Spring Data's `Pageable` and a custom `PageResponse<T>` DTO
- The API exposes this as `GET /auctions/search?q=&status=&category=&page=0&size=10`

4.4.5 Object Storage with MinIO

Item images are stored in **MinIO**, an S3-compatible object storage server, containerized via Docker Compose. The `ImageStorageService`:

1. **Auto-initializes** the bucket on startup (`@PostConstruct`), creating it and setting a public-read S3 policy if it doesn't exist
2. **Uploads** images with UUID-generated filenames to avoid collisions
3. **Returns** a direct public URL for the image

```java
// ImageStorageService.java - Upload flow
public String uploadImage(MultipartFile file) {
    String filename = UUID.randomUUID().toString() + extension;
    minioClient.putObject(PutObjectArgs.builder()
        .bucket(bucketName)
        .object(filename)
        .stream(is, file.getSize(), -1)
        .contentType(file.getContentType())
        .build());
    return externalUrl + "/" + bucketName + "/" + filename;
}
```

The Docker Compose configuration spins up MinIO alongside PostgreSQL, with a dedicated `createbuckets` init container that configures the `auction-images` bucket with public anonymous access.

```yaml
# docker-compose.yml
minio:
  image: minio/minio:RELEASE.2024-05-10T01-41-38Z
  ports: ["9000:9000", "9001:9001"]
  command: server /data --console-address ":9001"

createbuckets:
  image: minio/mc:RELEASE.2024-05-09T17-04-24Z
  entrypoint: >
    /bin/sh -c "
    mc alias set myminio http://minio:9000 minioadmin minioadmin;
    mc mb myminio/auction-images || true;
    mc anonymous set public myminio/auction-images;
    "
```

This approach keeps image storage fully separate from the relational database, allows horizontal scaling of storage independently, and provides direct HTTP access to images without proxying through the application server.